

Relatório de Atividades: Desenvolvimento de Web Service com Spring Boot

Disciplina: Sistemas Distribuídos

Instituição: UFC - Campus Quixadá

Autor(a): Gabriely Correia Dealem

1. Introdução e Objetivo

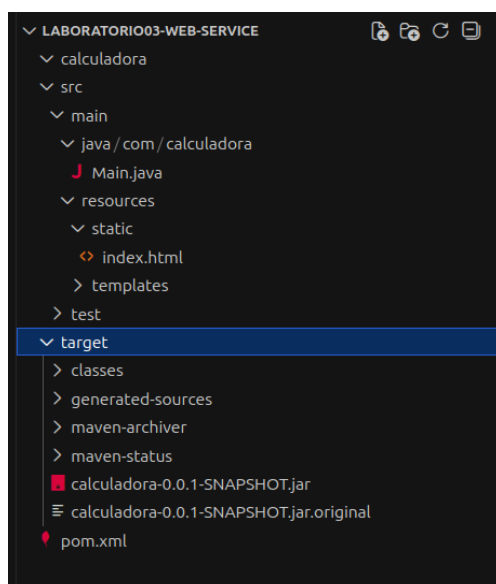
Esta atividade consistiu na criação e execução de um Web Service utilizando o framework **Spring Boot 2.1.6**. O foco principal foi a configuração de um ambiente de microserviço capaz de processar operações através de um servidor embarcado. O objetivo final foi a compilação do projeto via Maven e a inicialização bem-sucedida do servidor Tomcat integrado.

2. Tecnologias e Ferramentas

- **Linguagem:** Java 11 (ajustado para compatibilidade).
- **Framework:** Spring Boot 2.1.6.RELEASE.
- **Servidor de Aplicação:** Apache Tomcat 9 (Embarcado).
- **Gerenciador de Build:** Maven.

3. Arquitetura e Estrutura do Projeto

A aplicação foi estruturada seguindo o padrão Spring Boot, onde a classe principal (**Main.java**) gerencia o ciclo de vida da aplicação e as dependências são resolvidas automaticamente via *Starters* do Spring.

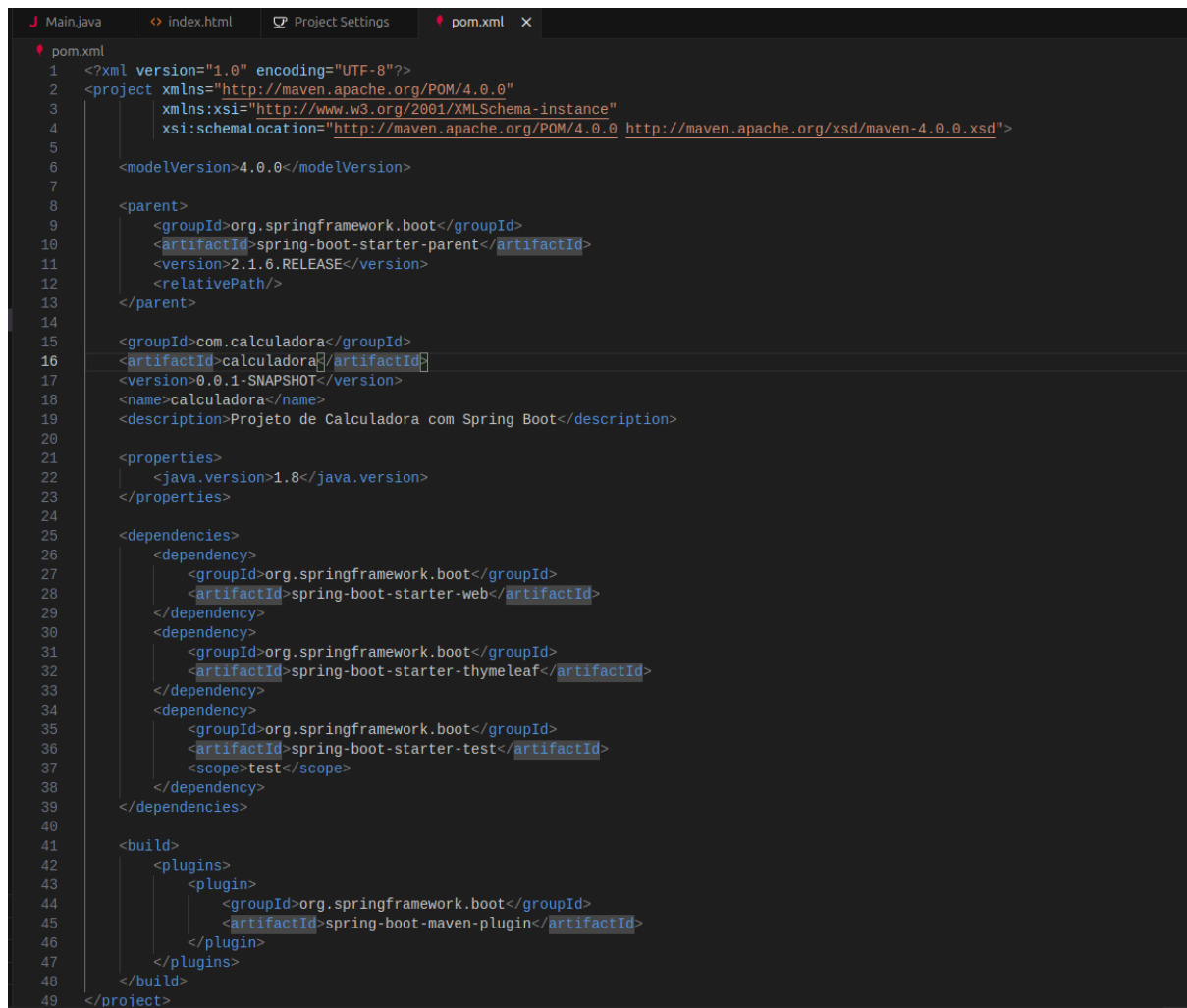


4. Implementação da Arquitetura

O projeto foi segmentado em três pilares fundamentais para garantir a execução:

4.1. Configuração do Maven (pom.xml)

O arquivo `pom.xml` foi configurado para utilizar o `spring-boot-starter-parent` na versão **2.1.6.RELEASE**. Foi necessária uma correção técnica na tag `<relativePath/>` para garantir que a resolução de dependências do Maven seguisse o padrão de repositório remoto.



```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5
6     <modelVersion>4.0.0</modelVersion>
7
8     <parent>
9         <groupId>org.springframework.boot</groupId>
10        <artifactId>spring-boot-starter-parent</artifactId>
11        <version>2.1.6.RELEASE</version>
12        <relativePath/>
13    </parent>
14
15    <groupId>com.calculadora</groupId>
16    <artifactId>calculadora</artifactId>
17    <version>0.0.1-SNAPSHOT</version>
18    <name>calculadora</name>
19    <description>Projeto de Calculadora com Spring Boot</description>
20
21    <properties>
22        <java.version>1.8</java.version>
23    </properties>
24
25    <dependencies>
26        <dependency>
27            <groupId>org.springframework.boot</groupId>
28            <artifactId>spring-boot-starter-web</artifactId>
29        </dependency>
30        <dependency>
31            <groupId>org.springframework.boot</groupId>
32            <artifactId>spring-boot-starter-thymeleaf</artifactId>
33        </dependency>
34        <dependency>
35            <groupId>org.springframework.boot</groupId>
36            <artifactId>spring-boot-starter-test</artifactId>
37            <scope>test</scope>
38        </dependency>
39    </dependencies>
40
41    <build>
42        <plugins>
43            <plugin>
44                <groupId>org.springframework.boot</groupId>
45                <artifactId>spring-boot-maven-plugin</artifactId>
46            </plugin>
47        </plugins>
48    </build>
49 </project>
```

4.2. Classe Principal

A classe `com.calculadora.Main` foi utilizada como ponto de entrada, anotada com `@SpringBootApplication`, permitindo o auto-scan de componentes e a configuração automática do servidor web.

```
J Main.java X index.html Project Settings pom.xml
src > main > java > com > calculadora > J Main.java
1 package com.calculadora;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5 import org.springframework.web.bind.annotation.GetMapping;
6 import org.springframework.web.bind.annotation.PathVariable;
7 import org.springframework.web.bind.annotation.RestController;
8
9 @RestController
10 @SpringBootApplication
11 public class Main {
12
13     @GetMapping("/somar/{a}/{b}")
14     public String somar(@PathVariable double a, @PathVariable double b) {
15         return "Resultado: " + (a + b);
16     }
17
18     @GetMapping("/subtrair/{a}/{b}")
19     public String subtrair(@PathVariable double a, @PathVariable double b) {
20         return "Resultado: " + (a - b);
21     }
22
23     @GetMapping("/multiplicar/{a}/{b}")
24     public String multiplicar(@PathVariable double a, @PathVariable double b) {
25         return "Resultado: " + (a * b);
26     }
27
28     @GetMapping("/dividir/{a}/{b}")
29     public String dividir(@PathVariable double a, @PathVariable double b) {
30         if (b == 0) {
31             return "Erro: Divisão por zero!";
32         }
33         return "Resultado: " + (a / b);
34     }
35
36     public static void main(String[] args) {
37         SpringApplication.run(Main.class, args);
38     }
39 }
```

5. Procedimentos de Build e Deploy

A construção foi realizada através do ciclo de vida do Maven. Devido a conflitos de versão, a compilação exigiu o uso explícito do Java 11 para evitar falhas no Tomcat.

Comandos utilizados:

- `JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn clean package`
- `JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn spring-boot:run`

Superação de Desafios Técnicos

O maior desafio desta etapa foi a incompatibilidade entre o **Spring Boot 2.1.6** e o **Java 17**. O erro `NoClassDefFoundError` na classe `MBeanFactory` do Tomcat foi identificado como uma restrição de acesso a módulos internos do Java moderno. A solução consistiu na reconfiguração do ambiente para **Java 11**, garantindo que o servidor Tomcat 9 pudesse inicializar seus componentes de gerenciamento (MBeans) sem restrições.

6. Resultados e Validação

A aplicação foi validada através dos logs de execução e do acesso à interface cliente:

- **Log de Inicialização:** `Tomcat initialized with port(s): 8080 (http)`
- **Acesso Local:** O serviço e a interface podem ser acessados via:
`http://localhost:8080`
- **Integração:** A interface HTML consome o Web Service via API Fetch, enviando os dados para o servidor e exibindo o resultado em tempo real.

Nota sobre o desempenho: A aplicação iniciou em aproximadamente **6.54 segundos**, tempo que pode variar conforme a disponibilidade de recursos de hardware do sistema.

7. Conclusão

A prática evidenciou que, em sistemas distribuídos, a compatibilidade entre o framework e o ambiente de execução (JVM) é crítica. A integração bem-sucedida entre o front-end HTML/JS e o back-end Spring Boot via `localhost:8080` demonstra o funcionamento pleno do modelo cliente-servidor proposto.